

오라이봇

오라이봇

- 건설현장 유도원 자동화 시스템 -

ROKEY BootCamp

(곽혜린, 권도원, 양준호, 윤재현, 임지현, 성형석, 정제유, 조의연, 진성호, 최찬우)

- 목차 -

1. 소개.....	
2. 시스템 개요.....	
3. 비즈니스 모델 목표 및 기대효과.....	
4. 종속성 및 제약조건.....	
5. 위험 완화조건.....	
6. 기능 요구사항.....	
7. 비기능 요구사항.....	
8. 성능 요구사항.....	
9. 시스템 아키텍처.....	
10. 구성 요소 설계.....	
11. 데이터 흐름 설계.....	
12. 네비게이션 서브시스템 설계.....	
13. 위험 탐지 서브시스템 설계.....	
14. 상세 설계.....	
15. 데이터 저장 및 로그 기록.....	
16. 보안 설계.....	
17. 테스트 및 오류 처리.....	
18. 배포 및 유지보수 계획.....	

1. 소개

건설 현장은 대형 중장비와 작업자가 동시에 작업하는 환경으로, 후진 및 이동 과정에서 발생하는 충돌 사고 위험이 높다. 특히 장비의 사각지대와 제한된 시야로 인해 작업자와의 사고가 빈번하게 발생하며, 이를 예방하기 위해 유도원이 배치되고 있다. 그러나 인력 중심 방식은 피로 누적, 숙련도 편차, 사각지대 한계 등으로 인해 완전한 안전을 보장하기 어렵고, 인건비 부담 또한 크다. 이에 본 보고서에서는 실시간 객체 인식과 자율주행 기능을 갖춘 로봇을 활용하여 기존 유도 방식을 보조하고, 건설 현장의 안전성과 효율성을 향상시키는 시스템을 제안한다.

2. 시스템 개요

본 시스템은 두 대의 모바일 로봇과 객체 인식, 자율주행, 관제 기능으로 구성된 중장비 안전 유도 보조 시스템이다. 로봇은 협력적으로 중장비 주변을 순회하며 작업자 및 장애물을 실시간으로 탐지하여 사각지대를 보완하고, 장비 이동 시 안전한 경로를 생성하여 유도 기능을 수행한다. 이동 중 위험 요소가 감지되면 즉각적인 경고와 경로 재설정을 통해 충돌을 방지하며, 관제 시스템을 통해 로봇 상태를 실시간으로 모니터링하고 필요 시 원격 제어가 가능하도록 설계되었다. 이를 통해 기존 인력 중심 방식의 한계를 보완하고, 보다 안정적인 안전 관리가 가능하도록 한다.

3. 비즈니스 모델 목표 및 기대효과

- **프로젝트 목표** : 건설 현장 중장비 이동 시 작업자 충돌 사고 감소를 위한 로봇 기반 안전 유도 시스템 구축 및 사각지대·유도 미흡 문제 해결
- **사고 현황** : 건설업 사고 사망자의 약 20%가 건설기계 관련 사고이며, 후진 및 이동 과정에서의 충돌 사고 비율 높은 구조
- **기존 방식 한계** : 유도원 중심 운영 방식의 피로 누적, 숙련도 편차, 사각지대 완전 커버 한계 및 안전성 확보의 어려움
- **비용 부담** : 유도원 1인 기준 일일 약 15만~20만원, 연간 약 4,500만~6,000만원 수준 인건비 발생 및 장비당 최대 약 1억 원 이상 비용 발생 구조
- **시스템 적용** : 실시간 객체 인식 및 자율주행 기반 2대 로봇 활용 작업 환경 모니터링, 안전 경로 생성, 위험 요소 감지 및 즉각적 경고·대응 수행

- **운영 효율성** : 작업자 피로도 및 숙련도 영향 없는 일정 성능 유지 기반 안정적 안전 관리 체계 확보
 - **안전성 향상** : 사각지대 기반 안전 대책 적용 시 사고 0 건 사례 기반 유사 수준 사고 감소 효과 기대
 - **경제적 효과** : 유도 인력 일부 대체 및 효율적 운영을 통한 연간 수천만 원 이상 인건비 절감
 - **간접 비용 절감** : 사고로 인한 공사 중단, 보상, 법적 책임 등 수천만~수억 원 규모 비용 예방 효과
 - **기대 효과** : 기존 인력 중심 방식 대비 안전성 향상 및 비용 절감 동시 달성, 건설 현장 효율적 안전 관리 체계 구축 기여
-

4. 종속성 및 제약조건

- **종속성**
 - ROS 기반 로봇 플랫폼 및 LiDAR, 카메라 등 센서 장비 의존
 - 로봇 간 및 관제 시스템 간 통신을 위한 안정적인 무선 네트워크 환경 필요
 - **제약조건**
 - 조명, 날씨, 먼지 등 환경 조건에 따른 객체 인식 성능 영향
 - 지형 상태에 따른 로봇 주행 성능 제한
 - 완전 자율주행이 아닌 보조 시스템 구조로서 원격 제어 필요성 존재
-

5. 위험 완화조건

- **객체 인식 오탐지 및 미탐지**
 - 연속 프레임 기반 검증 및 신뢰도 임계값 적용을 통한 탐지 정확도 향상
- **통신 장애로 인한 제어 불가**
 - 통신 끊김 시 자동 정지 모드 전환 및 최소 안전 기능 유지 설계
- **센서 오류 또는 데이터 불일치**
 - 다중 센서 기반 인식 및 이상 데이터 필터링을 통한 시스템 안정성 확보

- **자율주행 오류 및 경로 이탈**
 - 실시간 경로 재탐색 및 장애물 회피 적용, 필요 시 원격 제어 개입
 - **배터리 부족으로 인한 시스템 중단**
 - 배터리 상태 실시간 모니터링 및 임계치 이하 시 자동 복귀 또는 경고 수행
 - **작업자와의 근접 충돌 위험**
 - 일정 거리 이내 접근 시 즉각 정지 및 경고 신호 발생
-

6. 기능 요구사항

1. 자율 네비게이션

- **경로 생성 및 이동** : 작업 환경 내 중장비 이동 경로를 고려한 안전 경로 생성 및 자율 이동
- **동적 장애물 회피** : 작업자 및 자재 등 동적 장애물의 실시간 인식 및 충돌 방지를 위한 경로 수정 또는 정지 수행
- **협력 주행(다중로봇)** : 로봇 간 위치 정보 공유 및 중장비 주변 영역 분담을 통한 협력 이동 및 사각지대 최소화

2. 감시 및 모니터링

- **실시간 환경 인식** : 카메라 및 센서를 활용한 주변 환경 데이터 수집 및 작업자·장애물 정보 지속 업데이트
- **상태 모니터링** : 로봇 위치, 배터리 상태, 동작 상태 등의 실시간 관제 시스템 전송
- **이미지 전송** : 카메라 기반 캡처 이미지의 주기적 관제 시스템 전송 및 현장 상황 확인 지원

3. 위협 탐지 기능

- **객체 인식** : 사람, 차량, 장애물 등의 실시간 탐지 및 위험 요소 식별
- **위험 상황 판단** : 객체 위치 및 이동 상태 기반 충돌 가능성 분석 및 위험 상황 판별
- **탐지 신뢰도 보정** : 연속 프레임 검증 및 임계값 설정을 통한 오탐지 최소화 및 탐지 신뢰도 향상

4. 통신 및 경고 시스템

- **운전자 경고 전달** : 위험 상황 발생 시 스피커 기반 청각적 경고 신호 출력
- **시스템 간 통신** : 로봇, 관제 시스템, 사용자 인터페이스 간 실시간 데이터 송수신 및 통신 안정성 유지
- **비상 대응 기능** : 긴급 상황 발생 시 로봇 즉시 정지, 경고 강화 및 관제 시스템 경고 신호 전송

5. 사용자 인터페이스 (UI)

- **정보 제공** : 배터리 상태 및 웹캠 이미지 등 로봇 상태 정보 제공
 - **이벤트 및 로그 관리** : 시스템 이벤트 및 로그 정보 기록 및 사용자 조회 기능 제공
-

7. 비기능 요구사항

1. 성능 요구사항

- **실시간 처리 성능** : 객체 인식 및 위험 판단의 실시간 수행 및 입력 데이터부터 결과 출력까지 지연 최소화
- **응답시간** : 위험 상황 발생 시 경고 신호의 즉각적 전달 및 전체 시스템 반응 시간의 수 초 이내 유지
- **처리 안정성** : 연속 데이터 입력 환경에서 성능 저하 없는 안정적 동작 유지

2. 신뢰성 및 안정성

- **시스템 안정성** : 장시간 운영 환경에서 시스템 중단 없는 안정적 동작 유지
- **오류 대응** : 센서 오류 및 통신 장애 발생 시 안전 상태(정지 또는 경고) 전환
- **이중 안전 구조** : 자율주행 시스템과 원격 제어 시스템 병행 구성 및 수동 개입 가능성 확보

3. 정확도

- **객체 인식 정확도** : 사람, 차량, 장애물 등의 높은 정확도 탐지 및 오탐지·미탐지 최소화
- **위치 및 경로 정확도** : 지정 경로 이탈 없는 정확한 이동 및 장애물 회피 시 안정적 경로 유지

4. 확장성

- **시스템 확장성**: 로봇 및 센서 추가 시 기존 구조 유지 기반 확장 가능성 확보
- **모듈화 구조**: 인지, 판단, 제어, 통신 기능의 독립적 모듈 구성 및 유지보수·기능 확장 용이성 확보

5. 통신 및 네트워크

- **실시간 데이터 전송**: 로봇과 관제 시스템 간 지연 없는 안정적 데이터 전송
- **통신 안정성**: 통신 장애 상황에서도 최소 기능(정지, 경고) 유지

6. 사용성

- **직관적인 인터페이스**: 전문 지식 없이 사용 가능한 사용자 인터페이스 제공
- **경고 가시성**: 작업 환경에서 명확한 인지를 위한 시각적·청각적 경고 전달

7. 유지보수성

- **시스템 점검 용이성**: 전문 지식 없이 사용 가능한 사용자 인터페이스 제공
 - **로그 관리**: 시스템 동작 및 오류 로그 기록 및 분석·개선 활용 가능성 확보
-

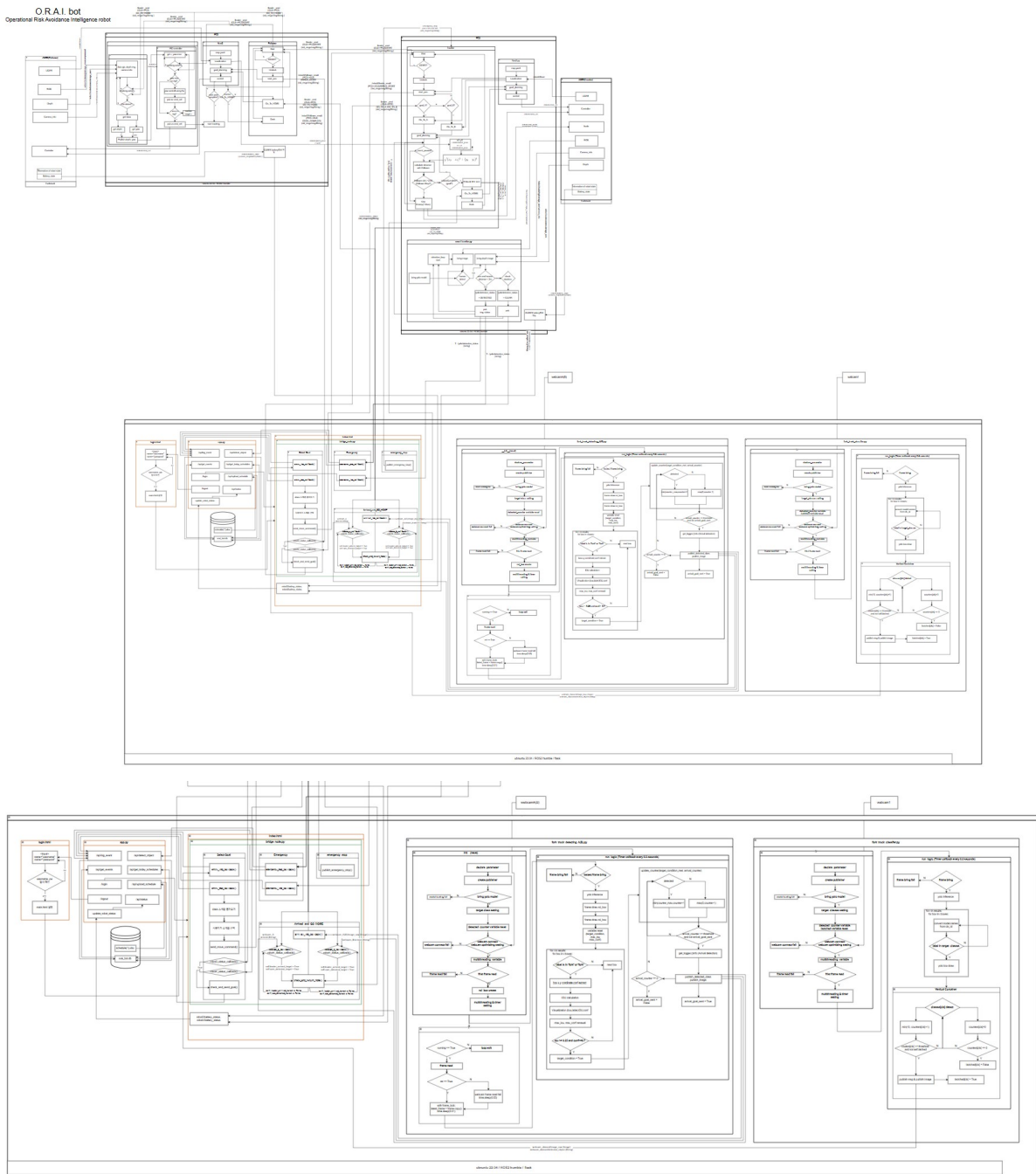
8. 성능 요구사항

- **실시간 처리**: 객체 인식 및 위험 판단의 실시간 수행 및 지연 없는 결과 반영
 - **응답 속도**: 위험 상황 발생 시 경고 및 제어 신호의 수 초 이내 전달
 - **안정성**: 장시간 운영 환경에서 성능 저하 없는 지속적 동작 유지
 - **정확도**: 객체 인식 및 거리 판단의 높은 정확도 유지 및 오탐지·미탐지 최소화
-

9. 시스템 아키텍처

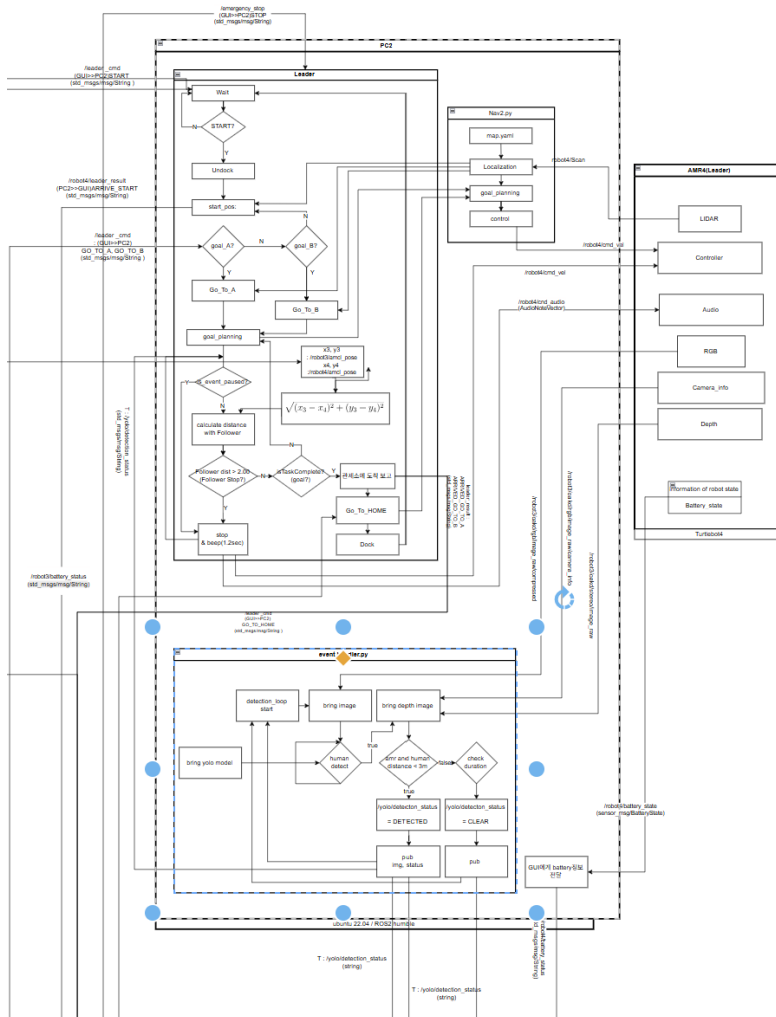
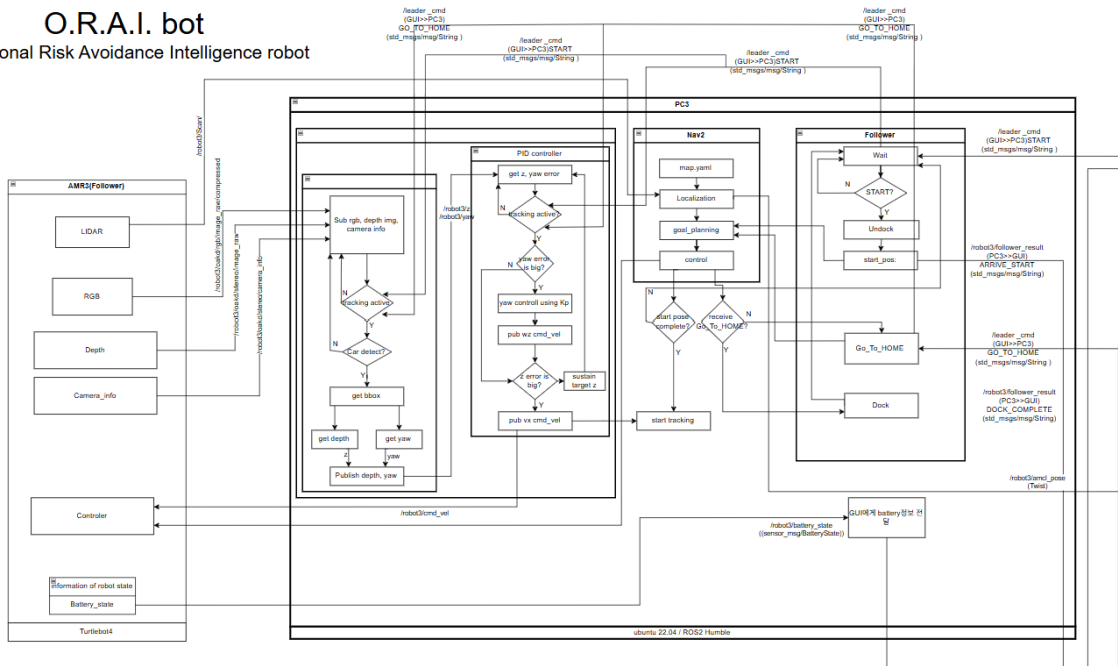
- **시스템 구조**: 두 대 TurtleBot4 기반 AMR 및 관제 PC 로 구성된 분산 구조 설계
- **환경 인식**: 각 로봇의 OAK-D Pro 카메라 및 LiDAR 센서 기반 주변 환경 인식
- **자율주행 및 객체 탐지**: ROS2 기반 객체 탐지 및 자율주행 수행
- **데이터 전송**: 객체 정보 및 로봇 상태 데이터의 ROS2 통신 기반 관제 PC 전송
- **데이터 처리 및 시각화**: 관제 PC 기반 데이터 처리 및 웹 대시보드 시각화
- **협력 동작**: 로봇 간 데이터 공유 기반 협력 동작 및 사각지대 최소화
- **안전 대응**: 위험 상황 발생 시 실시간 경고 및 제어 수행

ORAI bot
Operational Risk Avoidance Intelligence robot



O.R.A.I. bot

Operational Risk Avoidance Intelligence robot



10. 구성 요소 설계

1. 하드웨어 구성 요소

- **PC (모니터링 시스템)**
 - **운영 체제** : Ubuntu 22.04
 - **카메라** : USB 카메라(웹캠)
 - **네트워크** : Wi-Fi 6 를 통한 고대역폭 무선 연결 지원
- **자율 이동 로봇 (AMR) - TurtleBot4**
 - **프로세서** : Raspberry Pi 프로세서 탑재
 - **운영 체제** : Ubuntu 22.04 및 ROS2 (로봇 제어)
 - **센서** :
 - **LiDAR** : 네비게이션 및 장애물 감지
 - **USB 카메라** : TurtleBot4 의 위치 감지용 비디오 피드 캡처
 - **초음파 센서** : 근거리 장애물 회피
 - **Depth** :
 - OAK-D-PRO
 - 4K RGB auto focus camera (IMX378)
 - Mono stereo camera pair (OV9282) I
 - MU, Spatial AI processor
 - IR Laser Dot Projector & Illumination LED
 - **배터리** : 최소 2 시간 배터리 수명, 자율 도킹 스테이션
- **충전 스테이션**
 - 수동 개입 없이 AMR 의 충전을 위한 자율 도킹 스테이션

2. 소프트웨어 구성 요소

- **PC 소프트웨어**
 - **Python3** : 관제 시스템 및 데이터 처리 로직 구현을 위한 기본 언어
 - **ROS2** : 로봇과의 통신 및 데이터 송수신을 위한 미들웨어 플랫폼
 - **Flask** : 로봇 상태 모니터링 및 제어를 위한 웹 기반 대시보드 서버
 - **SQL** : 이벤트 로그 및 경고 기록을 저장하기 위한 경량 데이터베이스
 - **OpenCV** : 영상 데이터 처리 및 시각화 기능 수행

- **Ultralytics (YOLO)** : 객체 인식 결과 처리 및 추가 분석
 - **AMR 소프트웨어 (TurtleBot4 기반)**
 - **ROS2** : 로봇 제어, 센서 데이터 처리, 노드 간 통신을 위한 핵심 플랫폼
 - **Python3** : 로봇 동작 제어 및 객체 인식 연동을 위한 주요 개발 언어
 - **OpenCV** : 카메라 영상 처리 및 객체 데이터 전처리
 - **Ultralytics (YOLO)** : 실시간 객체 탐지 및 위험 상황 분석
 - **DepthAI SDK** : OAK-D Pro 카메라 제어 및 RGB/Depth 데이터 처리
 - **Navigation2 (Nav2)** : 자율주행, 경로 생성 및 이동 제어 기능 수행
-

11. 데이터 흐름 설계

- **데이터 수집**
 - **AMR 센서** : OAK-D Pro 카메라(RGB + Depth), LiDAR, IMU 기반 주변 환경 영상 및 거리 데이터 실시간 수집
 - **다중 로봇 구성** : 두 대 로봇의 독립적 데이터 수집 및 사각지대 최소화 구조
- **데이터 처리 및 분석**
 - **온보드 처리** : 온보드 컴퓨팅 환경 기반 객체 인식(YOLO) 및 센서 데이터 활용 작업자·차량·장애물 탐지 및 위험 여부 판단
 - **협력 판단** : ROS2 기반 로봇 간 정보 공유 및 협력 판단 수행
- **데이터 저장**
 - **AMR 로컬 저장** : 실시간 처리 성능 유지를 위한 최소 상태 정보 및 임시 데이터 저장
 - **PC 로그 저장** : SQL 기반 객체 탐지 결과, 경고 발생 기록, 시스템 상태 로그 저장
- **경고 및 알림**
 - **실시간 경고 전송** : 위험 상황 감지 시 ROS2 통신 기반 즉시 경고 신호 전송 및 LED·음성 신호 출력

- **대시보드 표시** : Flask 기반 웹 대시보드의 경고 및 상태 정보 실시간 표시 및 운영자 모니터링·개입 지원
-

12. 네비게이션 서브시스템 설계

1. 네비게이션 요구사항

- **자율 경로 계획** : 작업 구역 내 중장비 이동 경로 고려 안전 주행 경로 생성 및 자율 이동
- **실시간 장애물 회피** : 작업자 및 자재 등 동적 장애물 실시간 인식 및 충돌 방지를 위한 경로 수정·정지
- **위치 파악** : LiDAR 기반 SLAM 활용 로봇 위치 지속 추정 및 지도 상 정확한 위치 정보 유지
- **협력 주행** : 로봇 간 위치 정보 공유 기반 작업 구역 분담 탐색 및 사각지대 최소화

2. 아키텍처

- **SLAM 모듈** : LiDAR 데이터 기반 환경 지도 생성 및 로봇 위치 추정
- **경로 계획 모듈** : 목표 지점까지 최적 경로 생성 및 Nav2 기반 글로벌·로컬 플래너 활용
- **장애물 감지 및 회피 모듈** : LiDAR·카메라 데이터 기반 장애물 감지 및 로컬 경로 수정 기반 충돌 방지
- **협력 제어 모듈** : 로봇 간 위치·상태 정보 공유 기반 중복 영역 감소 및 효율적 탐색·유도 수행

3. 네비게이션 데이터 흐름

- **입력 데이터** : LiDAR 거리 데이터, 카메라 영상(OAK-D), IMU 데이터, 목표 위치 정보
- **프로세스 흐름**
 - **SLAM 처리** : 센서 데이터를 기반으로 실시간 위치 추정 및 지도 생성 수행
 - **경로 계획** : 목표 지점까지의 글로벌 경로 생성 및 로컬 경로 보정

- **장애물 감지** : 센서 데이터를 통해 정적 및 동적 장애물 탐지
 - **장애물 회피** : 탐지된 장애물을 기반으로 수정하거나 정지 수행
 - **협력 정보 공유** : 로봇 간 위치 및 상태 데이터를 공유하여 탐색 영역 분산 및 충돌 방지
 - **출력 데이터** : 이동 명령(속도, 방향), 경로정보, 로봇 위치 데이터
-

13. 위협 탐지 서브시스템 설계

1. 위협 탐지 요구사항

- **객체 탐지** : 작업자, 차량, 장애물 등 위험 요소의 실시간 탐지
- **실시간 처리** : 카메라 입력 데이터 기반 지연 없는 객체 인식 및 위험 판단
- **이상 로그 기록** : 탐지 객체 및 위험 상황의 로그 저장 및 사후 분석 가능성 확보
- **즉각적인 알림 전송** : 위험 상황 발생 시 로봇 및 관제 시스템 대상 즉각적 경고 전달

2. 아키텍처

- **이미지 캡처 모듈** : OAK-D Pro 카메라 기반 RGB·Depth 영상 실시간 수집
- **객체 탐지 모듈** : YOLO 기반 객체 탐지 및 위치·종류 분류 수행
- **이상 탐지 모듈** : 객체 위치·거리 정보 기반 충돌 위험 판단 및 임계값 초과 시 위험 상황 분류, 연속 프레임 검증 기반 오탐지 감소

3. 위협 탐지 데이터 흐름

- **입력 데이터** : 카메라 영상(RGB), Depth 데이터, 로봇 위치 정보
- **프로세스 흐름**
 - **이미지 전처리** : 해상도 조정 및 노이즈 제거 기반 입력 영상 전처리
 - **객체 탐지(YOLO)** : 객체 탐지 및 바운딩 박스·클래스 정보 생성
 - **이상 평가** : Depth 기반 객체-로봇 간 거리 계산 및 위험 기준 기반 충돌 가능성 판단
 - **로그 기록** : 탐지 결과 및 위험 이벤트를 로컬 또는 PC 에 저장

- **알림 전송** : 위험 상황 발생 시 ROS2 통신 기반 경고 신호 전송 및 시각·청각 장치 기반 즉각적 경고 수행
 - **출력 데이터** : 객체 정보(위치, 종류), 위험 여부, 경고 신호
-

14. 상세 설계

1. AMR 네비게이션 및 위험 탐지

- **네비게이션** : ROS 기반 Nav2 활용 작업 구역 내 자율주행 수행 및 LiDAR·센서 데이터 기반 경로 생성·추종
- **협력 주행** : 두 대 로봇의 협력 이동 및 중장비 주변 분담 탐색을 통한 사각지대 최소화
- **위험 탐지** : OAK-D Pro 카메라 및 YOLO 기반 객체 인식 활용 작업자·차량·장애물 실시간 탐지
- **위험 판단** : Depth 기반 거리 정보 활용 충돌 위험 여부 판단
- **통신 프로토콜** : ROS2 DDS 기반 로봇 간 및 PC 간 데이터 송수신 수행 및 객체 정보·위치 데이터·경고 신호 실시간 공유

2. 사용자 인터페이스 (대시보드)

- **모니터링 대시보드 (Flask 기반)** : Flask 기반 웹 대시보드 활용 로봇 위치·상태·객체 탐지 결과·경고 정보 실시간 시각화 및 시스템 상태 직관적 확인 지원
 - **수동제어** : 비상 상황 시 대시보드 기반 로봇 원격 제어 수행 및 자율주행 한계 상황에서의 운영자 개입 지원
-

15. 데이터 저장 및 로그 기록

- **AMR 로컬 저장** : 실시간 객체 인식 및 주행 처리를 위한 최소 상태 정보 (위치, 속도 등) 및 임시 데이터의 로컬 저장, 장기 데이터 저장 미수행
- **PC 로그 저장** : SQL 기반 객체 탐지 결과(사람, 차량), 로봇 상태 정보, 경고 발생 시점 및 유형 저장

- **이벤트 기반 이미지 저장** : 위험 상황 발생 시 OAK-D Pro 카메라 이미지 선택적 저장 및 이벤트 중심 데이터 기록
 - **ROS2 데이터 기록** : 로봇 간 및 PC 통신 과정 주요 토픽 데이터의 선택적 기록 및 시스템 분석·디버깅 활용
 - **로그 관리** : 시간 기준 로그 관리 및 위험 발생·경고 등 이벤트 중심 조회 구조 구성
 - **데이터 활용** : 사고 상황 분석, 객체 인식 성능 개선, 시스템 안정성 검증 등 후속 연구 및 시스템 개선 활용
-

16. 보안 설계

- **데이터 암호화** : 로봇과 PC 간 통신 데이터의 암호화된 통신 방식 적용 및 데이터 유출·변조 방지
 - **접근 제어** : 관제 시스템 및 대시보드의 사용자 인증 기반 접근 제한 및 승인 사용자 대상 모니터링·제어 기능 제공
 - **로컬 저장 보안** : PC 저장 로그 및 이벤트 데이터의 외부 접근 제한 환경 관리 및 최소 정보 저장 기반 데이터 노출 방지
-

17. 테스트 및 오류 처리

1. AMR Part

- 본 장에서는 자율주행 및 군집 주행 과정에서 발생 가능한 예외 상황 정의 및 시스템 안정성 확보를 위한 소프트웨어적 대응 방안 구성

1.1. 리더(Leader) 로봇

- **홈 복귀 시퀀스 최적화** : GO_TO_HOME 수행 전 도킹 상태 확인 및 언도킹 이후 이동 수행, 물리적 간섭 방지
- **다중 이벤트 중첩 제어** : 상태 플래그(is_event_paused) 및 재시작 쿨타임(goal_start_time) 기반 이벤트 동기화 및 재주행 안정성 확보
- **초기 위치 이동 및 탐지 순서 변경** : 초기 위치 도달 이후 이벤트 탐지 활성화 기반 탐지 누락 방지

- **비전 기반 장애물 인식 고도화** : LiDAR 한계 보완을 위한 OAK-D 기반 비전 장애물 탐지 적용 및 비상 정지(Emergency Stop) 수행 구조
- **비전 기반 YOLO 및 뎀스(Depth) 측정 도입** : OAK-D 카메라 및 YOLO 기반 객체 분류 및 Depth 데이터 기반 거리 측정 수행
 - **객체 인식** : YOLO 기반 대상 객체(사람, 로봇 등) 정밀 분류
 - **거리 측정** : Depth 데이터 기반 객체까지의 물리적 거리 산출 및 지능형 이벤트 처리 적용
- **이벤트 처리 및 거리 기반 제어**
 - **이벤트 발생 구역 선정** : 차량 유도 상황에서만 이벤트 필요 조건 반영 기반 안내 구간(GO_TO_A, GO_TO_B)에서만 이벤트 처리 수행 및 시작 준비·복귀 구간 이벤트 비활성화
 - **기존 문제** : START 이동 및 복귀 과정에서 장애물로 인한 리더-팔로워 간 거리 2.0m 이상 증가 시 목표 좌표 미도달 문제
 - **개선 효과** : 불필요한 이벤트 제거 및 경로 이동 안정성 확보
- **팔로워(Follower) 로봇 거리 동기 제어**
 - **거리 계산** : 리더·팔로워 로봇의 amcl_pose 기반 실시간 거리 산출
 - **정지 조건** : 로봇 간 거리 2.0m(max_f_dist) 초과 시 경로 정지 이벤트 발생
 - **재개 조건** : 거리 1.5m(resum_f_dist) 이하 접근 시 이벤트 해제 및 주행 재개
 - **제어 목표** : 로봇 간 거리 1.5~2.0m 범위 유지 기반 안정적 차량 유도 및 군집 주행 안정성 확보

1.2. 팔로워(Follower) 로봇

- **주행 제어 파라미터 튜닝**
 - **속도 제어 응답성 개선** : 거리 비례 제어 상수($0.5 \rightarrow 0.75$) 조정 기반 리더 속도 변화 대응성 향상
 - **안전 속도 및 오차 범위 설정** : 최대 속도 0.45m/s 제한 및 유지 거리 0.7m, 허용 오차 0.3m 설정 기반 안정적 군집 주행 확보
- **데이터 전송 및 처리 부하 경감**

- **영상 데이터 규격 최적화** : 프레임 15fps, 해상도 640p 설정 기반
연산 부하 감소
- **압축 전송 방식 채택** : Compressed Image 기반 토픽 송수신 및
네트워크 대역폭 절감·지연 최소화
- **리더 동기화 기반 자동 복귀 시퀀스**
 - **명령 전파 및 상태 동기화** : 리더 복귀 명령 수신 시 통신 노드 기반
상태 정보 전파
 - **순차적 복귀 수행** : 팔로워의 복귀 모드 전환 및 리더 궤적·속도 동기화
기반 안전 복귀
- **동일 신호 기반 상태 동기화**
 - **기존 구조** : leader_cmd, follower_cmd 분리 토픽 기반 개별 명령
송신
 - **개선 방안** : 단일 토픽(leader_cmd) 기반 동일 명령 동시 송신 구조
적용
 - **기대 효과** : 리더·팔로워 로봇 간 상태 동기화 확보 및 협업 주행
안정성 향상
- **상태 전송 방식을 주기 발행에서 이벤트 기반으로 변경**
 - **이벤트 기반 상태 발행** : 상태 변경 시점 기반 follower_state
발행 및 불필요 토픽 감소
 - **상태 전환 명확성 확보** : 상태 변화 시점 중심 publish 구조 적용
- **MultiThreadExecutor 와 callback group 유지**
 - **MultiThreadExecutor 적용** : 명령 수신 및 상태 감시 병렬 처리
기반 응답성 향상
 - **MutuallyExclusiveCallbackGroup 적용** : 콜백 간 간섭 방지 및
안정적 실행 구조 확보
- **충돌 방지를 위한 추종 활성화 조건 및 안전 제어 로직 개선**
 - **추종 시작 조건 제한** : “ARRIVED_START” 수신 이후 tracking_act
ive 활성화 기반 초기 충돌 방지

- 복귀 시 추종 차단 : “GO_TO_HOME” 수신 시 tracking 중지 및 정지 명령 발행 기반 Nav2 동작 충돌 방지
- 데이터 지연 대응을 위한 안전 정지 로직
 - 데이터 유효성 검증 : z, yaw 수신 시간 기반 timeout(0.5s) 초과 시 제어 중단
 - 이상 데이터 차단 : 거리 0 이하, NaN 등 비정상 값 감지 시 즉시 정지 처리
- YOLO 처리 조건 최적화 및 거리 추정 안정화
 - 조건부 객체 검출 : tracking_active 스위칭으로 필요 시에만 YOLO 수행 및 연산 부하 감소
 - 거리 계산 안정화 : 중심 주변 영역(5×5) 기반 percentile 적용으로 depth 노이즈 감소

2. YOLO Part

- AMR 환경 객체 인식 과정에서의 오검출 및 인식 불안정 문제 정의 및 모델 선정 · 데이터 기반 성능 개선 방안 구성
- 2.1. 초기 모델 비교 및 선정
 - 모델 비교 : YOLOv8n, YOLOv8m, YOLOv8l 모델 대상 데이터 증강 미적용 상태 성능 비교
 - 성능 평가 : 검증 데이터 기준 mAP@0.5 = 0.995, F1-score = 1.00 기반 기본 검출 성능 확보
 - 실시간성 평가 : inference time 비교 결과 YOLOv8m(약 11.05 ms/image) 모델의 우수한 처리 속도 확인
 - 모델 선정 : 실시간 객체 검출 및 추적 요구 충족을 위한 YOLOv8m 모델 우선 선정
- 2.2. 실제 AMR 환경 적용 시 문제점
 - 오검출 발생 : TurtleBot4, 배경 구조물, 바닥 표시 등 비검출 대상 객체 검출 문제

- **bounding box 불안정** : 로봇 이동 및 회전에 따른 객체 인식 영역 불안정 발생
- **오인식 문제** : 객체가 다른 물체로 잘못 인식되는 현상 발생

2.3. 데이터셋 재구성

- **오검출 환경 데이터 추가**
 - 실제 AMR 환경 기반 오탐 발생 데이터 수집 및 다양한 각도·조명 조건 반영
- **선택적 라벨링 적용**
 - truck, fork 대상 bounding box 지정 및 기타 객체 라벨링 제외 기반 비검출 대상 학습 배제 구조

2.4. 추가 학습 결과

- **성능 유지** : mAP@0.5 약 0.995 수준 유지 및 Precision·Recall 높은 수준 확보
- **실시간성 확보** : inference time 약 13.62 ms/image 수준 유지
- **성능 개선** : 오검출 감소 및 bounding box 안정성 향상

2.5. 최종 모델 선정

- **성능 개선** : 실제 환경 데이터 반영 학습 기반 검출 안정성 및 신뢰성 향상
- **최종 선정** : 실제 운용 환경 기준 최적화된 YOLOv8m 모델 선정

2.6. 데이터 증강 미사용

- **증강 제외** : 데이터 증강 적용 시 성능 저하 발생
- **학습 전략** : 실제 환경 기반 원본 데이터 중심 학습 수행

3. GUI Part

3.1. 시스템 안정성 및 데이터 신뢰성 확보

- **이미지 데이터 처리 최적화** : CvBridge() 기반 이미지 인코딩·디코딩 적용 및 numpy 수동 변환 제거를 통한 데이터 왜곡 방지 및 런타임 에러 대응력 향상
- **파일 시스템 접근 신뢰성** : 상대 경로 대신 절대 경로 고정 사용 기반 노드 실행 위치와 무관한 안정적 데이터 저장 및 데이터 유실 방지

3.2. 복합 조건부 종료

- **기존 문제** : 단순 위치 정보 기반 종료 시 센서 오차 및 환경 요인으로 인한 작업 미완료 상태 종료 위험
- **개선 방안** : ‘위치 도착’ 및 ‘객체 인식’ 조건의 AND 논리 적용 기반 작업 종료 조건 강화
- **기대 효과** : 조건 미충족 시 대기 및 재시도 수행 기반 작업 정확도 향상 및 논리적 안전성 확보

3.3. 협업 로봇(Multi-Robot) 동기화 및 제어

- **기존 문제** : 리더·팔로워 간 개별 명령 수행으로 인한 지연 발생 시 충돌 및 경로 이탈 위험
- **개선 방안** : robot3_ready, robot4_ready 기반 독립 상태 플래그 관리 및 상호 교차 검증 구조
- **안정성 확보** : 전체 로봇 준비 완료 시에만 다음 단계 수행 기반 통신 지연 대응 및 협업 주행 안정성 확보

3.4. 테스트 및 검증

- **단위 테스트**
 - **데이터 파싱 테스트** : 엑셀 업로드 데이터의 SQLite schedules 테이블 저장 여부 검증
 - **상태 모니터링 테스트** : CPU, RAM 및 로봇 상태 데이터 정상 반환 여부 검증
 - **이미지 처리 테스트** : CvBridge 기반 Image → OpenCV 변환 및 저장 정상 동작 검증
 - **이벤트 로그 테스트** : 이벤트 발생 시 logs.html 기반 UI 기록 정상 출력 여부 검증
- **사용자 승인 테스트 (UAT)**
 - **관리자 시나리오** : login 기반 접속 후 작업 스케줄 업로드 및 로봇 작업 현황 모니터링 사용성 검증
 - **일정 반영 검증** : 엑셀 업로드 기반 당일 작업 일정의 정확한 화면 표시 확인

- **작업 흐름 검증** : 입차 감지 → 일정 선택 → 출발 확정까지의 조작 흐름 직관성 평가
- **예외 처리 시나리오** : 수동 지정 구역 출발 기능 활용 미등록 차량 처리 및 이력 기록 검증
- **비상 대응 UI 평가** : 비상 정지 버튼 위치·크기·색상 기반 긴급 상황 인지 가능성 평가

4. 핵심 성능 평가 결과

- 본 시스템에 대한 반복 주행, 객체 인식, 장애 대응 테스트를 수행 한 결과 다음과 같은 성능 확인
 - **제어 안정성** : 99.5%%
 - **객체 인식 정확도** : 98.2%
 - **시스템 복구 시간** : 0.5 초 이내
 - 이를 통해 자율주행 제어 안정성, 위험 탐지 신뢰성, 장애 대응 성능 측면에서 현장 적용 가능성을 확인
-

18. 배포 및 유지보수 계획

1. GUI Part

- **배포 방식** : Flask 기반 웹 서버 형태로 관제 PC 에 배포 및 로컬 네트워크 환경에서 접근 가능한 구조
- **실행 환경** : Python 기반 런타임 환경 구성 및 필수 라이브러리(Flask, SQL 등) 의존성 관리
- **업데이트 방식** : 서버 코드 수정 후 재배포 및 서비스 재시작 방식 적용
- **로그 관리** : 이벤트 기반 로그 저장 및 주기적 데이터 정리(백업 및 불필요 데이터 삭제) 수행
- **유지보수 계획**
 - UI 개선 및 기능 확장 시 모듈 단위 수정
 - 사용자 피드백 기반 인터페이스 개선
 - 로그 기반 오류 분석 및 시스템 개선 반영

2. AMR Part

- **배포 방식** : ROS2 패키지 형태로 각 TurtleBot4 로봇에 배포 및 노드 단위 실행 구조
- **실행 환경** : ROS2 Humble 기반 환경 구성 및 Nav2, 센서 드라이버, 커스텀 노드 포함
- **업데이트 방식**
 - 노드 단위 코드 수정 후 재빌드(colcon build) 및 재실행
 - 주요 파라미터(속도, 거리 등) 설정 파일 기반 동적 수정
- **통신 관리** : ROS2 DDS 기반 네트워크 환경 구성 및 로봇 간 통신 상태 지속 모니터링
- **유지보수 계획**
 - 주행 로그 및 토픽 데이터 기반 오류 분석
 - 이벤트 로직 및 제어 파라미터 지속 튜닝
 - 센서(LiDAR, 카메라) 상태 점검 및 캘리브레이션 수행

3. YOLO Part

- **배포 방식** : 학습된 YOLOv8m 모델을 온보드 환경에 배포하여 실시간 추론 수행
- **실행 환경** : Python 기반 딥러닝 추론 환경 구성 및 GPU/CPU 환경 최적화 적용
- **업데이트 방식**
 - 신규 데이터 수집 후 모델 재학습
 - 성능 검증 후 모델 파일 교체 방식으로 업데이트
- **데이터 관리**
 - 실제 환경 데이터 지속 수집 및 데이터셋 확장
 - 라벨링 데이터 품질 관리 및 버전 관리 수행
- **유지보수 계획**
 - 오검출 사례 기반 데이터셋 보완
 - 모델 성능 지표(mAP, Precision, Recall) 지속 모니터링
 - 환경 변화(조명, 배경 등)에 따른 재학습 및 성능 개선